Deep Reinforcement Learning

Policy Gradients

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

- From value-based methods to direct policy optimization
- Policy gradients
  - The objective
  - Evaluation the objective
  - Differentiating the objective
- The REINFORCE algorithm
- Understanding and challenges
- Reducing the variance
  - Reward to go
  - Baselines

Where are we?

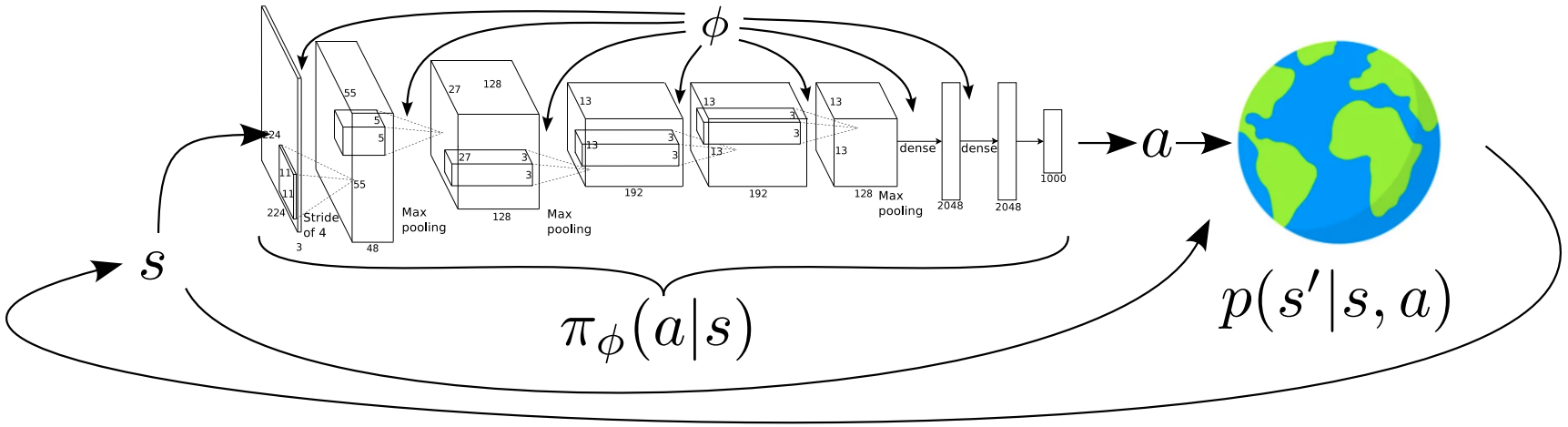
Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	Value estimation with function approximation
8	Deep Q-learning	Q-learning with neural networks
9	Policy gradients	Direct optimization of the policy
10	Actor-critic algorithms	
11	Advanced algorithms	
	Model-Based Control	
	Advanced Topics	

Lecture contents

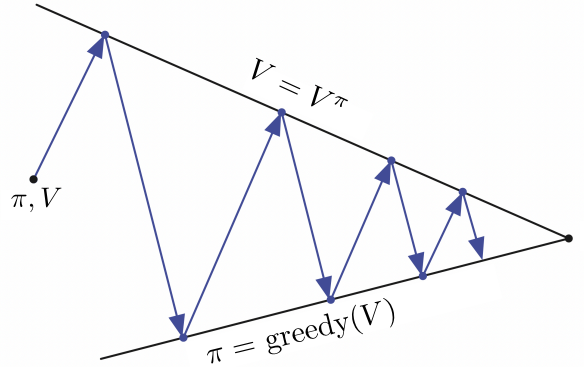
From value-based methods to direct policy optimization

From value-based methods to direct policy optimization

- Until now, all approaches followed the GPI concept.
- These are all **value-based methods**: Need an estimate of V^π or Q^π to improve π.
- But: what are we after in RL, really? $\Rightarrow$ The **optimal policy** π^*!



Inspired by Sergey Levine's CS285 lecture.



Value-based methods: Generalized Policy Iteration (
(Sutton and Barto 1998).

Alternative approach:

1. Define probabilities over entire trajectories.

$$\Rightarrow \quad p_\phi(\underbrace{s_0, a_0, \ldots, s_{T-1}, a_{T-1}, s_T}_{=\tau}) = p_\phi(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_\phi(a_t|s_t) p(s_{t+1}|s_t, a_t).$$

2. Formulate RL objective; optimize dynamics via policy parameters ϕ.

$$\Rightarrow \quad \phi^* = \arg\max_\phi \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} r_t \right].$$

The reinforcement learning objective

$$\phi^* = \arg\max_\phi \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} r_t \right]$$

Infinite horizon case: $\phi^* = \arg\max_\phi \mathbb{E}_{(s,a) \sim p_\phi(s,a)} \left[r \right]$

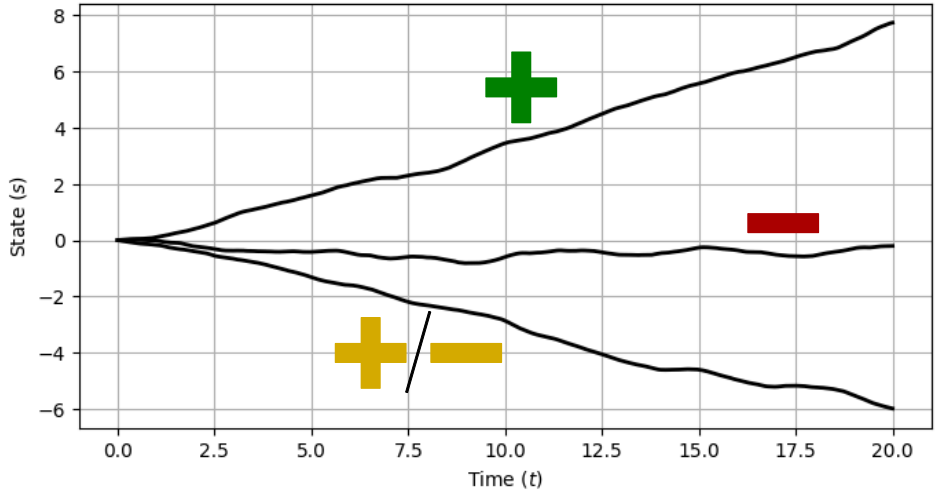
Finite horizon case: $\phi^* = \arg\max_\phi \sum_{t=0}^{T-1} \mathbb{E}_{(s_t,a_t) \sim p_\phi(s_t,a_t)} \left[r_t \right]$

- Infinite horizon: expected reward according to the stationary distributions of s and a.
- Finite horizon: linearity $\Rightarrow$ expected reward of individual stages (may differ with t).
- We are sticking to the finite-horizon case here.

💡 We are not considering the discount factor γ for now.

Evaluating the objective

$$\phi^* = \arg\max_{\phi} \underbrace{\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} r_t \right]}_{L(\phi)}$$



Inspired by Sergey Levine's CS285 lecture.

- First, let's think about evaluating the $L(\phi)$ for a fixed policy π.
- How do we estimate expectations if we don't have access to a model?

$\Rightarrow$ Monte Carlo sampling!

$$L(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} r_t \right] \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} r_{i,t}$$

- Sample N trajectories with s_0 according to the initial distribution and then following π.
- Take the sample average over these trajectories.

Policy gradient: Depending on the return, the plan is to increase or reduce the trajectories' likelihoods by adapting the policy π.

Differentiating the policy (1)

$$\phi^* = \arg\max_\phi \underbrace{\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\sum_{t=0}^{T-1} r_t\right]}_{L(\phi)}$$

$$L(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}[\underbrace{r(\tau)}_{=\sum_{t=0}^{T-1} r_t}] = \int p_\phi(\tau) r(\tau)\,\mathrm{d}\tau$$

$$\nabla_\phi L(\phi) = \nabla_\phi \left[\int p_\phi(\tau) r(\tau)\,\mathrm{d}\tau\right]$$

$$= \int \nabla_\phi p_\phi(\tau) r(\tau)\,\mathrm{d}\tau$$

$$= \int p_\phi(\tau)\nabla_\phi \log p_\phi(\tau) r(\tau)$$

$$= \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\nabla_\phi \log p_\phi(\tau) r(\tau)\right] \quad (1)$$

Definition of the expectation in continuous spaces
The expected value of a function $x(\tau)$ is

$$\mathbb{E}_{\tau\sim p}\left[x(\tau)\right] = \int p(\tau) x(\tau)\,\mathrm{d}\tau.$$

Here, p is the density according to which τ is distributed, with $\int p(\tau)\,\mathrm{d}\tau = 1$.

Note: Both integration and differentiation are *linear* operations
$\Rightarrow$ We can swap!

A convenient identity

$$p_\phi(\tau)\nabla_\phi \log p_\phi(\tau) = p_\phi(\tau)\frac{\nabla_\phi p_\phi(\tau)}{p_\phi(\tau)} = \nabla_\phi p_\phi(\tau) \quad (2)$$

Differentiating the policy (2)

$$\phi^* = \arg\max_\phi L(\phi)$$

$$L(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[r(\tau)\right] = \int p_\phi(\tau) r(\tau)\,\mathrm{d}\tau$$

$$\nabla_\phi L(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\nabla_\phi \log p_\phi(\tau) r(\tau)\right]$$

$$\underbrace{p_\phi(s_0, a_0, \ldots, s_{T-1}, a_{T-1}, s_T)}_{=p_\phi(\tau)} = p(s_0)\prod_{t=0}^{T-1}\pi_\phi(a_t|s_t)p(s_{t+1}|s_t,a_t)$$

take log on both sides! ⇓ $(\log(a\cdot b) = \log(a) + \log(b))$

$$\log p_\phi(\tau) = \log p(s_0) + \sum_{t=0}^{T-1}\left[\log \pi_\phi(a_t|s_t) + \log p(s_{t+1}|s_t,a_t)\right]$$

$$\begin{aligned}
\nabla_\phi \log p_\phi(\tau) &= \nabla_\phi\left[\log p(s_0) + \sum_{t=0}^{T-1}\left[\log \pi_\phi(a_t|s_t) + \log p(s_{t+1}|s_t,a_t)\right]\right] \\
&= \nabla_\phi \log p(s_0) + \sum_{t=0}^{T-1}\left[\nabla_\phi \log \pi_\phi(a_t|s_t) + \nabla_\phi \log p(s_{t+1}|s_t,a_t)\right] \\
&= \cancel{\nabla_\phi \log p(s_0)} + \sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t) + \cancel{\nabla_\phi \log p(s_{t+1}|s_t,a_t)}
\end{aligned}$$

Policy gradient theorem (simplified)

$$\begin{aligned}
\nabla_\phi L(\phi) &= \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t)\right) r(\tau)\right] \\
&= \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t)\right)\left(\sum_{t=0}^{T-1} r_t\right)\right]
\end{aligned}$$

Policy gradient theorem

What does this mean?

1. We want to find a policy that maximizes the value:

$$\max_{\pi} \mathbb{E}_{\tau \sim p^{\pi}} \left[\sum_{t=0}^{T-1} r_t \right] = \max_{\pi} \mathbb{E}_{\tau \sim p^{\pi}} \left[r(\tau) \right].$$

Policy gradient theorem (simplified)

$$\nabla_{\phi} L(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t | s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right]$$

2. Using a neural network approximator π_{ϕ} $\Rightarrow$ maximization w.r.t. the policy parameters: $\max_{\phi} \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[r(\tau) \right] = \max_{\phi} L(\phi)$.
3. This yields a challenging loss function to optimize: Integration over τ (i.e., the space of trajectories) is infeasible!
   $\Rightarrow$ Approximate via Monte Carlo sampling:

$$L(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[r(\tau) \right] = \int p_{\phi}(\tau) r(\tau) \, \mathrm{d}\tau \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} r_{i,t}.$$

4. Use $\log$ identity to derive a formulation of the gradient that we can *approximate using sampling*:

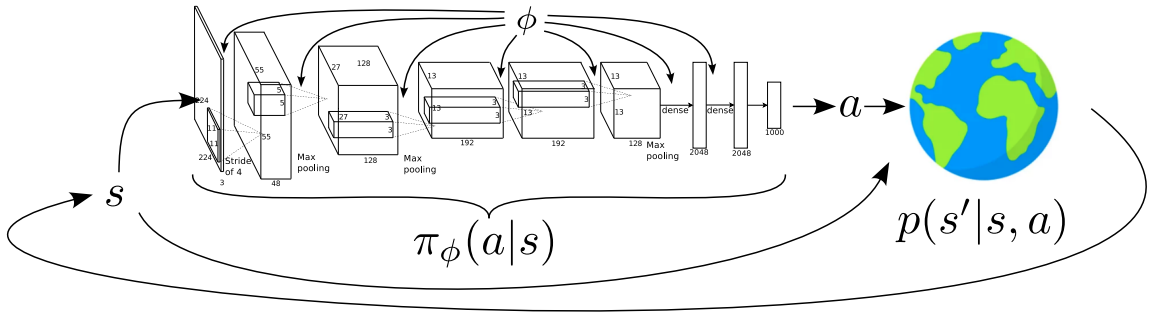
$$\nabla_{\phi} L(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t | s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right].$$

REINFORCE

We now have a formulation of the policy gradient that we can **approximate using Monte Carlo sampling**:

$$\nabla_\phi L(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right] \approx \frac{1}{N} \sum_{i=1}^{N} \underbrace{\left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \right)}_{\text{(I)}} \underbrace{\left(\sum_{t=0}^{T-1} r_{i,t} \right)}_{\text{(II)}}.$$

(I) This term is simpliy the derivative of the policy network w.r.t. the weights $\Rightarrow$ backpropagation!
(II) This term is the experience we collect from interactions with the environment.



The REINFORCE algorithm

1. Sample $\{\tau_i\}_{i=1}^N$ using $\pi_\phi(a|s) \Rightarrow \{((s_{i,0}, a_{i,0}, r_{i,0}), \ldots, (s_{i,T-1}, a_{i,T-1}, r_{i,T-1}))\}_{i=1}^N$.
2. $\nabla_\phi L(\phi) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \right) \left(\sum_{t=0}^{T-1} r_{i,t} \right)$.
3. Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L(\phi)$.

Understanding the policy gradient

Continuous action example: A Gaussian policy

- Let's assume that we have a Gaussian policy whose mean is modeled by a neural network:

$$\pi_\phi(a|s) = \mathcal{N}\left(f_{\text{NN}}(s), \Sigma\right) = \frac{1}{C}\exp\left(-\frac{1}{2}(a - f_{\text{NN}}(s))^\top \Sigma^{-1}(a - f_{\text{NN}}(s))\right) = \frac{1}{C}\exp\left(-\frac{1}{2}\|a - f_{\text{NN}}(s)\|_\Sigma^2\right).$$

- Taking the log yields (with $\log C^{-1} = -\log C$):

$$\log \pi_\phi(a|s) = -\frac{1}{2}\|a - f_{\text{NN}}(s)\|_\Sigma^2 - \log C.$$

- Taking the gradient leads to the following expression:

$$-\frac{1}{2}\Sigma^{-1}(f_{\text{NN}}(s) - a)\frac{\partial f_{\text{NN}}}{\partial \phi}.$$

The REINFORCE algorithm

1. Sample $\{\tau_i\}_{i=1}^N$ using $\pi_\phi(a|s) \Rightarrow \{((s_{i,0}, a_{i,0}, r_{i,0}), \ldots, (s_{i,T-1}, a_{i,T-1}, r_{i,T-1}))\}_{i=1}^N$.
2. $\nabla_\phi L(\phi) \approx \frac{1}{N}\sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\right)\left(\sum_{t=0}^{T-1} r_{i,t}\right)$.

3. Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L(\phi)$.

Comparison to maximum likelihood

- Consider the task of maximizing the likelihood L of a training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ in supervised learning, e.g., using a neural network with parameter vector θ:

$$\max_\theta L(\theta) = \max_\theta p_\theta(y_1|x_1) \times \ldots \times p_\theta(y_N|x_N)$$
$$= \max_\theta \prod_{i=1}^N p_\theta(y_i|x_i).$$

- Optimizing over products is hard $\Rightarrow$ take the log:

$$\log L(\theta) = \log \left(\prod_{i=1}^N p_\theta(y_i|x_i) \right) = \sum_{i=1}^N \log p_\theta(y_i|x_i).$$

Maximum likelihood gradient:

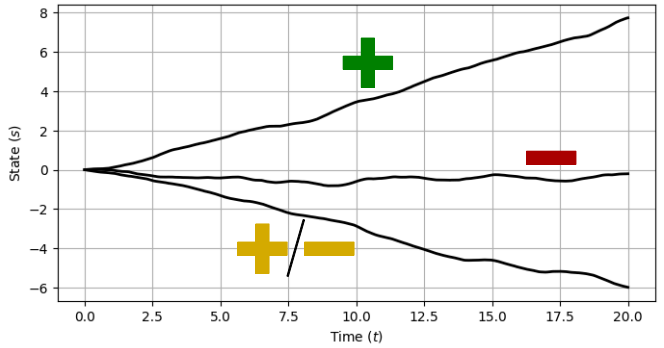
$$\nabla_\theta \log L(\theta) = \sum_{i=1}^N \log \nabla_\theta p_\theta(y_i|x_i).$$

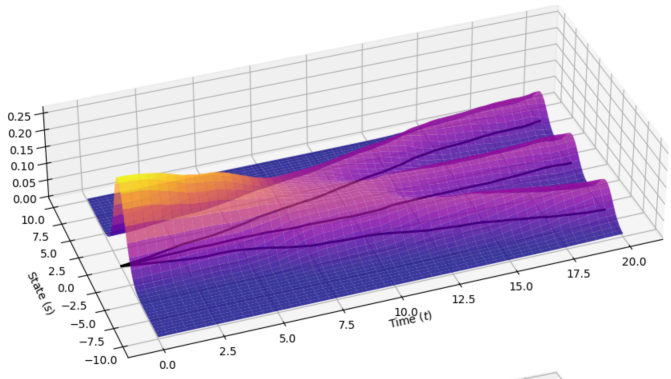
Comparison against the policy gradient:

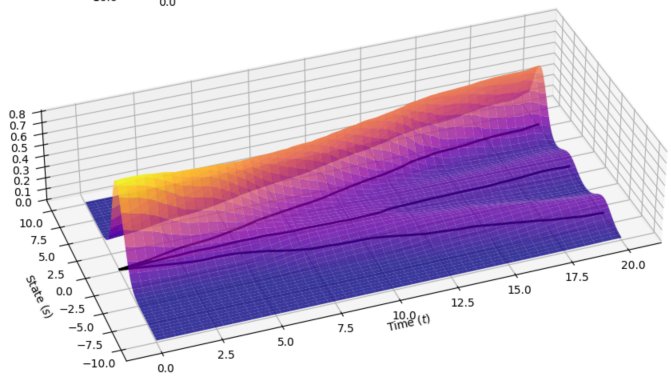
$$\nabla_\phi L(\phi) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \right) \left(\sum_{t=0}^{T-1} r_{i,t} \right).$$

- The policy gradient can be seen as a weighted version of the maximum likelihood objective.
- Why is weighting is necessary?
  - In maximum likelihood, we always want to maximize the likelihood of the "correct" labels.
  - In policy gradients, we may also want to reduce the likelihood of low-reward samples!

Understanding the policy gradient







- What did we just do?
  $\Rightarrow$ let's rewrite the formula a little and *compare against maixmum likelihood*:

$$\nabla_\phi L(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \underbrace{\nabla_\phi \log \pi_\phi(\tau_i)}_{\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})} \underbrace{r(\tau_i)}_{\sum_{t=0}^{T-1} r_{i,t}} \quad \text{vs.} \quad \nabla_\theta L(\theta) = \sum_{i=1}^{N} \log \nabla_\theta \pi_\phi(\tau_i).$$

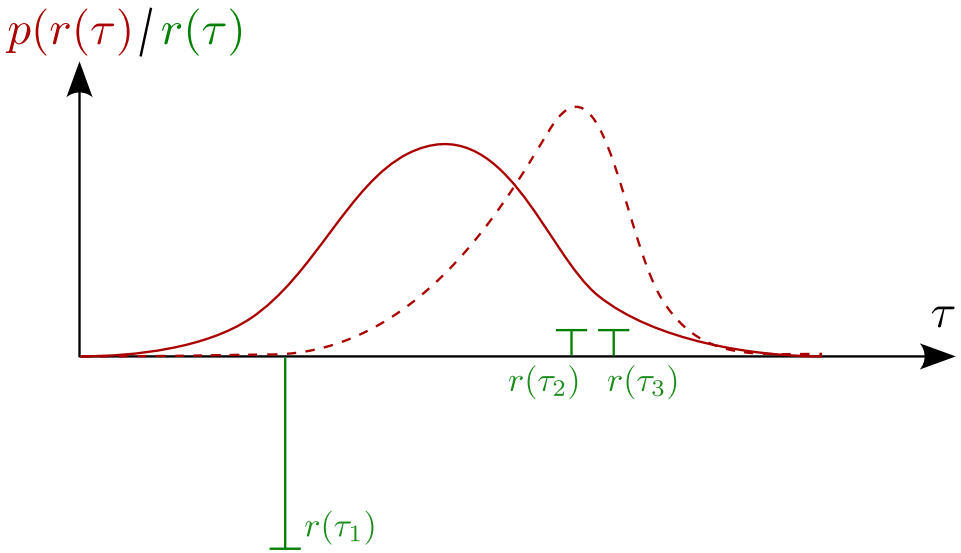
- Good experience is made more likely: We increase the proability of the policy to produce similar trajectories
- Bad experience is made less likely.
- This simply formalizes the notion of "trial and error"!

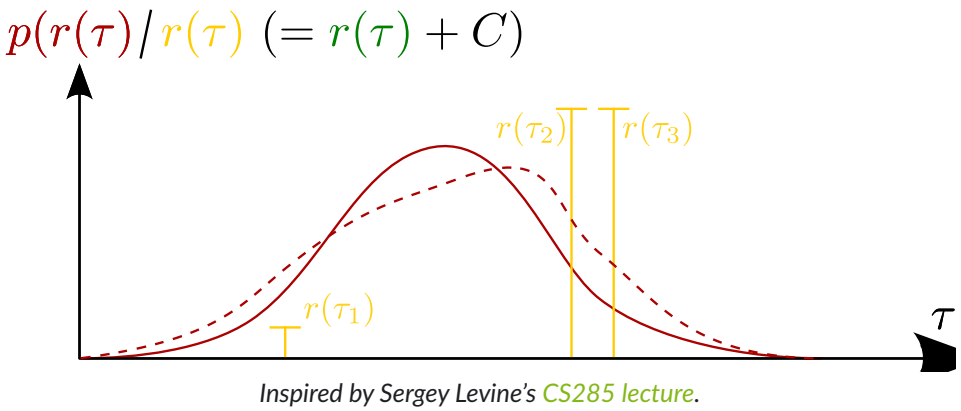
A note on partial observability (Without going into details)
The policy gradient also holds for partially observed MDPs (POMDPs). That is, for policies $\pi(a|o)$. In simple terms, the reason is that the policy gradient theorem does not make use of the Markov property.

Inspired by Sergey Levine's CS285 lecture.

Challenges with policy gradients

High variance





Inspired by Sergey Levine's CS285 lecture.

$$\nabla_\phi L(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\phi \log \pi_\phi(\tau_i) r(\tau_i).$$

- Adding a constant to the reward should not change the optimal policy!
  - This is true for *any* optimization problem, e.g.,

$$\arg\min_x f(x) = \arg\min_x [f(x) + 1000].$$

- In the limit $N \to \infty$, this is true for policy gradients as well…
  - … but it does not hold for finite sample sizes, in particular few sample trajectories.
- Depending on the sign of $r(\tau)$, we **either increase or decrease** the probability of seeing similar trajectories τ_i (and their rewards $r(\tau_i)$) in the future.
- This is an instance of the **high variance** issue with policy gradients.
- An even more "catastropic" version of this: What if we scale some of the rewards to be exactly zero?

Reducing variance – baselines

$$\nabla_\phi L(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\phi \log \pi_\phi(\tau_i) r(\tau_i).$$

- **Question**: Is there a systematic way to fix the challenge of reward offsets?
  $\Rightarrow$ Let's balance the "weight" of of our likelihood objective in such a way that ...
  - Better-than-average rewards *increase* the probability of the respective trajectories.
  - Worse-than-average rewards *decrease* the probability.
- Several **advantages**!
  - This aligns with our intuition that better-than-average is reinforced and worse-than-average is penalized.
  - This approach makes the policy gradient independent of the actual size of the reward signal.

Approach: subtract a **baseline** b from the reward signal

$$\nabla_\phi L(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\phi \log \pi_\phi(\tau_i) [r(\tau_i) - b],$$

where $b = \frac{1}{N} \sum_{i=1}^{N} r(\tau_i)$.

Theorem
Subtracting any constant b is *unbiased in expectation*

Proof: We start with the exact formulation of the policy gradient,

$$\nabla_\phi L(\phi) \overset{(1)}{=} \mathbb{E}_{\tau \sim p_\phi(\tau)} [\nabla_\phi \log p_\phi(\tau) r(\tau)].$$

Unbiased in expectation $\Rightarrow$ baseline term is zero in expectation:

$$\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\nabla_\phi \log p_\phi(\tau) r(\tau)\right] = \int p_\phi(\tau)\nabla_\phi \log p_\phi(\tau) b \,\mathrm{d}\tau$$

$$\overset{(2)}{=} \int \nabla_\phi p_\phi(\tau) b \,\mathrm{d}\tau = b\nabla_\phi \int p_\phi(\tau)\,\mathrm{d}\tau = (\nabla_\phi 1) b = 0. \qquad \square$$

The optimal baseline

$$\nabla_\phi L(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\nabla_\phi \log p_\phi(\tau) (r(\tau) - b) \right].$$

How do we find the optimal baseline?
$\Rightarrow$ optimization: minimize the variance

$$\text{var}\left[x\right] = \mathbb{E}\left[x^2\right] - \mathbb{E}[x]^2$$

$$\text{var} = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[(\underbrace{\nabla_\phi \log p_\phi(\tau)}_{=g(\tau)} (r(\tau) - b))^2 \right] - \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\nabla_\phi \log p_\phi(\tau) \left(r(\tau) - \cancel{b} \right) \right]^2 \quad \text{(unbiased baseline)}$$

$$\begin{aligned}
\frac{\mathrm{d}\text{var}}{\mathrm{d}b} &= \frac{\mathrm{d}}{\mathrm{d}b} \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 (r(\tau) - b)^2 \right] = \frac{\mathrm{d}}{\mathrm{d}b} \left(\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 r(\tau)^2 \right] - 2\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 r(\tau) b \right] + \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 b^2 \right] \right) \\
&= \frac{\mathrm{d}}{\mathrm{d}b} \left(\cancel{\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 r(\tau)^2 \right]} - 2b\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 r(\tau) \right] + b^2 \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 \right] \right) \\
&= -2\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 r(\tau) \right] + 2b\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 \right] \stackrel{!}{=} 0.
\end{aligned}$$

The optimal baseline b^* is the expected reward, *weighted by gradient magnitudes*:

$$b^* = \frac{\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 r(\tau) \right]}{\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[g(\tau)^2 \right]}.$$

Note: b^* is hard to calculate. In practice, we usually resort to the average reward

$$b = \frac{1}{N} \sum_{i=1}^{N} r(\tau_i).$$

Reducing variance – causality

$$\nabla_\phi L(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \right) \left(\sum_{t'=0}^{T-1} r_{i,t'} \right).$$

- **Causality**: The policy at time t' cannot affect the reward at time t when $t < t'$.
  - "What you do now, is not going to change the rewards you received in the past."
- **Question**: Are we making use of causality in the above equation?
  - Let's rewrite it and make use of the distributive property ($a \cdot (b + c) = a \cdot b + a \cdot c$):

$$\nabla_\phi L(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \left(\sum_{t'=0}^{T-1} r_{i,t'} \right).$$

    $\Rightarrow$ Past rewards (i.e., $t' < t$) have an impact on the policy π_ϕ!
  - In expectation, these factors have to cancel out (and one can prove this). **But**: for finite sample sizes, they do not and instead increase the variance.
- **Simple fix**: "*reward to go*" $\hat{Q}_{i,t} = \sum_{t'=t}^{T-1} r_{i,t'}$ (that is, the only change is $0 \to t$),

$$\nabla_\phi L(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \hat{Q}_{i,t}.$$

💡 Do not confuse causality with the Markov property! The Markov property (which may hold for a system, but does not have to) says that your future states do not depend on past states, just the present state. Causality is always true: "Rewards in the past are independent of decisions in the present."

Off-policy policy gradients

Advanced policy gradients

Covariant/natural PG

References

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.